Formal Verification of a Collatz Residue Sieve

A machine-checked correctness proof of `Residual_Analysis.py`

RobinCodes

github.com/RobinCodes/Collatz

17 August 2026

Abstract

We give a complete correctness proof, machine-checked in the Rocq Prover, of `Residual_Analysis.py` — a program that iteratively sieves residue classes modulo 2^q that cannot contain a minimal counterexample to the Collatz conjecture. Three aspects of the implementation are non-obvious and are the substance of the proof. First, the inner loop truncates its final right-shift at the step budget, producing an intermediate value that is not an element of the Collatz trajectory; we show it nevertheless equals $\mathrm{T}^q(n)$ exactly, where T is the accelerated Collatz map. Second, the program tests only the least positive representative of each residue class; we prove via a constructive form of Terras' theorem that this single test decides the whole class. Third, the program treats a fixed point $\mathrm{T}^q(r) = r$ as a *dead* class while separately recording it; we show this is sound, because every strictly larger member of such a class descends. The main theorem states that for every odd $n \geq 1$ whose class the sieve has discarded, either n reaches 1 given that all smaller positive integers do — so n is not a minimal counterexample — or n lies on a cycle, and every such n is reported in the program's `cycles` list. The Rocq development is axiom-free and contains no admitted proofs.

1 Introduction

The Collatz conjecture asserts that iterating

$$n \longmapsto \begin{cases} n/2 & n \text{ even}, \\ 3n+1 & n \text{ odd}, \end{cases}$$

from any positive integer eventually reaches 1. It remains open; computational searches have confirmed it for all n below roughly 2^{68}.

A standard partial attack is the *residue sieve* of Terras [1] and Everett [2]. Because the first q steps of the accelerated map depend only on n mod 2^q, one may partition $\mathbb{N}_{\geq 1}$ into 2^q residue classes and ask, for each class, whether *every* member drops below itself within q steps. A class with that property can contain no minimal counterexample and may be discarded; a surviving class is refined modulo 2^{q+1} and retested. Terras proved that the density of surviving classes tends to 0; the conjecture nevertheless remains open, because a density-zero set may still be nonempty.

The program `Residual_Analysis.py` in the repository github.com/RobinCodes/Collatz implements this sieve. Its arithmetic core is compact and, in three places, does something that looks wrong at first reading. This paper proves that it is right, and states precisely what it does and does not establish.

Contributions.

1. A precise specification of the algorithm (Section 3) and of the accelerated map's affine structure (Section 4).

2. A constructive *shift lemma* (Theorem 4.3) replacing the usual congruence formulation of Terras' theorem, which is what makes the Rocq development tractable.
3. Proofs that the truncated shift loop computes T^q (Proposition 3.2), that the least-representative test decides the class (Theorem 5.2), and that the sieve discards no potential minimal counterexample (Theorem 6.6).
4. A full Rocq formalization (Section 7), axiom-free, with the structural invariants that justify the program's reported density figure.
5. An explicit account of the algorithm's limits (Section 9).

2 The accelerated map

Definition 2.1 (Accelerated Collatz map)**.** $\mathrm{T} : \mathbb{N} \to \mathbb{N}$ is

$$\mathrm{T}(n) \;=\; \begin{cases} n/2 & n \text{ even}, \\ (3n+1)/2 & n \text{ odd}. \end{cases}$$

We write T^q for the q-fold iterate and set $\mathsf{reaches1}(n) \iff \exists k.\, \mathrm{T}^k(n) = 1$.

Since $3n+1$ is even for odd n, both branches are exact integer divisions. The following triviality is the reason the program can count *halvings* rather than *steps*.

Lemma 2.2 (Step accounting)**.** *Every application of* T *divides exactly once by* 2*. Consequently, for any n and q, the value obtained from n by repeatedly applying $m \mapsto 3m+1$ at odd values and dividing out powers of two, stopping the instant the* total *number of divisions by* 2 *reaches q, is $\mathrm{T}^q(n)$.*

Definition 2.3 (2-adic valuation)**.** For $y \neq 0$, $v_2(y)$ is the largest e with $2^e \mid y$.

Lemma 2.4 (Chained halving)**.** *Let m be odd and let $d \geq 1$ satisfy $2^d \mid 3m+1$. Then*

$$\mathrm{T}^d(m) \;=\; \frac{3m+1}{2^d}.$$

Proof. Induction on d. For $d = 1$: m is odd, so $\mathrm{T}(m) = (3m+1)/2$. Suppose the claim for d and let $2^{d+1} \mid 3m+1$; then also $2^d \mid 3m+1$, so $x := \mathrm{T}^d(m) = (3m+1)/2^d$ by the induction hypothesis. Writing $3m+1 = 2^{d+1}t$ gives $x = 2t$, which is even, hence $\mathrm{T}^{d+1}(m) = \mathrm{T}(x) = x/2 = (3m+1)/2^{d+1}$. □

Note that Lemma 2.4 does *not* require $d = v_2(3m+1)$. This is exactly the freedom the implementation exploits.

3 The program

3.1 Source

The arithmetic core of `Residual_Analysis.py` is:

```
def collatz_step(n: int, score: int) -> tuple[int, int]:
        y = 3 * n + 1
        v2 = (y & -y).bit_length() - 1

        d = min(v2, q - score)
        return y >> d, d

```

```
def collatz_sim(n: int) -> bool:
        og_n = n
        score = 0
        while score < q:
                data = collatz_step(n, score)
                n = data[0]
                score += data[1]

        if og_n < n:
                return True    # Survives
        elif og_n > n:
                return False   # Dies
        else:
                cycles.append(og_n)
                return False
```

3.2 Pseudocode

We abstract the two loops. Algorithm 1 is `collatz_sim`'s inner loop; Algorithm 2 is the outer sieve, including the reporting behaviour of the current version of the program.

Algorithm 1 EVAL(n, q) — evaluate $\mathrm{T}^q(n)$ for odd n by truncated shifting.

1: $score \leftarrow 0$; $\quad m \leftarrow n$
2: **while** $score < q$ **do**
3: $\quad y \leftarrow 3m + 1$ ▷ *y is even, since m is odd*
4: $\quad d \leftarrow \min(v_2(y),\ q - score)$ ▷ $1 \le d \le q - score$
5: $\quad m \leftarrow y \,/\, 2^d$ ▷ $y \gg d$
6: $\quad score \leftarrow score + d$
7: **return** m

Figure 1:

Algorithm 2 SIEVE — iterative residue elimination.

1: $q \leftarrow 1$; $\quad \mathcal{R} \leftarrow [\,1\,]$; $\quad \mathcal{C} \leftarrow [\,]$
2: **loop**
3: $\quad \mathcal{S} \leftarrow [\,]$
4: $\quad$ **for** $r \in \mathcal{R}$ **do**
5: $\quad\quad m \leftarrow \mathrm{EVAL}(r, q)$
6: $\quad\quad$ **if** $m > r$ **then** append r to $\mathcal{S}$ ▷ *class survives*
7: $\quad\quad$ **else if** $m = r$ **then** append r to $\mathcal{C}$ ▷ *cycle element; class still dies*
8: $\quad$ **report** $100 - 100\,|\mathcal{S}|/2^q$, $\quad q$, $\quad \mathcal{S}$ if $q \le 15$, $\quad \mathcal{C}$
9: $\quad \mathcal{R} \leftarrow [\,r,\ r + 2^q \ :\ r \in \mathcal{S}\,]$
10: $\quad q \leftarrow q + 1$

Figure 2:

Three features deserve comment, and each is discharged below.

(P1) Line 3 of Algorithm 1 applies $3m + 1$ unconditionally, with no parity test. This is safe only if m is odd on every entry to the loop body, which is not immediate: line 5 can leave m *even* when the budget truncates the shift.

(P2) Line 4 caps d at the remaining budget. When $d < v_2(y)$ the returned m is not a Collatz trajectory element at all. The comparison on line 6 of Algorithm 2 is nevertheless meaningful.

(P3) Only the least representative r of each class is ever tested, yet the conclusion drawn concerns the entire infinite class $\{\, r + j2^q : j \geq 0 \,\}$.

Proposition 3.1 (Loop invariant and (P1))**.** *Let n be odd and $q \geq 0$. Throughout Algorithm 1,*

$$m = \mathrm{T}^{score}(n) \qquad \textit{and} \qquad (\, m \textit{ is odd} \vee \textit{ score} = q \,).$$

Moreover $d \geq 1$ at every iteration, so the loop terminates after at most q iterations.

Proof. Initially $m = n = \mathrm{T}^0(n)$ is odd. Assume the invariant with $score < q$; then m is odd, so $y = 3m + 1$ is even and $v_2(y) \geq 1$, whence $d = \min(v_2(y), q - score) \geq 1$ and $d \leq v_2(y)$, so $2^d \mid y$. By Lemma 2.4, $y/2^d = \mathrm{T}^d(m) = \mathrm{T}^{score+d}(n)$, establishing the first conjunct. For the second: if $d = v_2(y)$ then $y/2^d$ is odd by maximality of the valuation; otherwise $d = q - score$, and the new score equals q. Since $d \geq 1$, the score strictly increases. □

Proposition 3.2 ((P2))**.** *For odd n and every $q \geq 0$, $\textsc{Eval}(n, q) = \mathrm{T}^q(n)$.*

Proof. Immediate from Proposition 3.1: the loop exits with $score = q$ — the score increases by $d \leq q - score$, so it never overshoots — and the invariant then reads $m = \mathrm{T}^q(n)$. □

Proposition 3.2 is the resolution of the truncation puzzle. The final value may be even and may not occur anywhere in the Collatz trajectory of n; it is nonetheless *exactly* $\mathrm{T}^q(n)$, because the truncated division is the tail of the q-th T-step, not a corruption of it.

4 Affine structure

Definition 4.1 (Affine coefficients)**.** Define $A_q(n), C_q(n) \in \mathbb{N}$ by recursion on q:

$$(A_0, C_0) = (1, 0), \qquad (A_{q+1}, C_{q+1}) = \begin{cases} (A_q,\ C_q) & \mathrm{T}^q(n) \text{ even}, \\ (3A_q,\ 3C_q + 2^q) & \mathrm{T}^q(n) \text{ odd}. \end{cases}$$

Lemma 4.2 (Affine identity)**.** *For all n, q:*

$$2^q\, \mathrm{T}^q(n) \;=\; A_q(n)\, n \;+\; C_q(n), \qquad A_q(n) = 3^{a_q(n)} \textit{ is odd}, \qquad C_q(n) \geq 0,$$

where $a_q(n)$ is the number of odd steps among the first q.

Proof. Induction on q. The base case is $n = n$. Write $m = \mathrm{T}^q(n)$ and assume $2^q m = A_q n + C_q$. If m is even, $\mathrm{T}^{q+1}(n) = m/2$ and $2^{q+1}(m/2) = 2^q m = A_q n + C_q$, matching the unchanged coefficients. If m is odd, $\mathrm{T}^{q+1}(n) = (3m+1)/2$ and $2^{q+1} \cdot \frac{3m+1}{2} = 2^q(3m+1) = 3(A_q n + C_q) + 2^q = (3A_q)n + (3C_q + 2^q)$. Oddness of A_q is immediate: 1 is odd and $3A_q$ is odd whenever A_q is. □

The next theorem is Terras' theorem in the form we need. The usual statement is a congruence — if $n \equiv n' \pmod{2^q}$ then the first q parities agree — but the constructive version below is both stronger and easier to formalize, since it computes the displacement rather than merely asserting equality of parity vectors.

Theorem 4.3 (Shift lemma)**.** *For all $q \in \mathbb{N}$ and all $n, j \in \mathbb{N}$,*

$$\mathrm{T}^q(n + j\, 2^q) \;=\; \mathrm{T}^q(n) + A_q(n)\, j, \qquad \textit{and} \qquad (A_q, C_q)(n + j\, 2^q) = (A_q, C_q)(n).$$

Proof. Simultaneous induction on q. For $q = 0$ both claims are trivial ($A_0 = 1$).

For the step, put $P = 2^q$ and note $n + j\,2^{q+1} = n + (2j)P$. Write $A = A_q(n)$ and $m = \mathrm{T}^q(n)$. The induction hypothesis applied with displacement $2j$ gives

$$\mathrm{T}^q\Big(n + j\,2^{q+1}\Big) = m + 2Aj, \qquad (A_q, C_q)\Big(n + j\,2^{q+1}\Big) = (A_q, C_q)(n).$$

Since $2Aj$ is even, $m + 2Aj$ has the same parity as m; hence the coefficient recursion of Definition 4.1 takes the same branch at step $q + 1$ for both arguments, which proves the second claim. For the first:

- m even: $\mathrm{T}(m+2Aj) = \dfrac{m + (Aj)\cdot 2}{2} = \dfrac{m}{2} + Aj = \mathrm{T}^{q+1}(n) + A_{q+1}(n)\,j$, using $A_{q+1}(n) = A$.
- m odd: $m + 2Aj$ is odd and $\mathrm{T}(m + 2Aj) = \dfrac{3m + 1 + (3Aj)\cdot 2}{2} = \dfrac{3m+1}{2} + 3Aj = \mathrm{T}^{q+1}(n) + A_{q+1}(n)\,j$, using $A_{q+1}(n) = 3A$.

In both cases the identity $(x + y\cdot 2)/2 = x/2 + y$ for exact integer division is all that is required. □

Corollary 4.4 (Class uniformity)**.** T^q *restricted to a residue class modulo* 2^q *is the affine map* $r + j2^q \mapsto \mathrm{T}^q(r) + A_q(r)\,j$. *In particular* $\mathrm{T}^q(n) - n$ *is monotone in* j*: strictly decreasing if* $A_q(r) < 2^q$ *and strictly increasing if* $A_q(r) > 2^q$.

5 The least-representative test

We now discharge (P3): why testing r alone suffices.

Lemma 5.1. *Let* $q \geq 1$ *and* $r \geq 1$ *with* $\mathrm{T}^q(r) \leq r$. *Then* $A_q(r) < 2^q$.

Proof. By Lemma 4.2, $A_q(r)\,r + C_q(r) = 2^q \mathrm{T}^q(r) \leq 2^q r$. Since $C_q(r) \geq 0$ we get $A_q(r)\,r \leq 2^q r$, and since $r \geq 1$, $A_q(r) \leq 2^q$. Equality is impossible: $A_q(r)$ is odd by Lemma 4.2, while 2^q is even for $q \geq 1$. Hence $A_q(r) < 2^q$. □

Theorem 5.2 (Descent)**.** *Let* $q \geq 1$ *and* $r \geq 1$ *with* $\mathrm{T}^q(r) \leq r$. *Then for every* $j \geq 0$ *such that* $\mathrm{T}^q(r) < r$ *or* $j \geq 1$,

$$\mathrm{T}^q(r + j\,2^q) \;<\; r + j\,2^q.$$

Proof. Write $A = A_q(r)$; by Lemma 5.1, $A < 2^q$. By Theorem 4.3, $\mathrm{T}^q(r + j2^q) = \mathrm{T}^q(r) + Aj$. If $\mathrm{T}^q(r) < r$ then $\mathrm{T}^q(r) + Aj < r + 2^q j$ since $Aj \leq 2^q j$. If instead $j \geq 1$ then $Aj < 2^q j$ strictly, and $\mathrm{T}^q(r) \leq r$ suffices. □

Theorem 5.2 covers both branches the program takes:

- **Strict descent** ($\mathrm{T}^q(r) < r$): the whole class descends, including r itself. The program discards it.
- **Fixed point** ($\mathrm{T}^q(r) = r$): every member *strictly above* r descends, but r itself does not — it lies on a cycle. The program discards the class and separately records r in `cycles`. This is sound, and it is why the equality branch is not a bug.

Remark 5.3. The fixed-point case has a pleasant reformulation. If $\mathrm{T}^q(r) = r$ then Lemma 4.2 gives $C_q(r) = (2^q - A_q(r))\,r$, so for $n = r + j2^q$,

$$\mathrm{T}^q(n) - n \;=\; \frac{(2^q - A_q(r))\,(r - n)}{2^q} \;<\; 0 \qquad (j \geq 1),$$

an explicit descent rate.

Theorem 5.4 (No minimal counterexample)**.** *Let $q \geq 1$, $r \geq 1$, $\mathrm{T}^q(r) \leq r$, and let $n = r + j2^q$ with $\mathrm{T}^q(r) < r$ or $j \geq 1$. If every m with $1 \leq m < n$ satisfies* $\mathsf{reaches1}(m)$*, then* $\mathsf{reaches1}(n)$*.*

Proof. By Theorem 5.2, $\mathrm{T}^q(n) < n$. Since $n \geq 1$ and T maps nonzero values to nonzero values, $\mathrm{T}^q(n) \geq 1$. By hypothesis $\mathsf{reaches1}(\mathrm{T}^q(n))$, say $\mathrm{T}^k(\mathrm{T}^q(n)) = 1$; then $\mathrm{T}^{k+q}(n) = 1$, so $\mathsf{reaches1}(n)$. □

6 The sieve

Definition 6.1. Let $\mathrm{survives}(q, r) \iff \mathrm{T}^q(r) > r$ and, for a list L,

$$\mathrm{children}_q(L) \;=\; [\, r,\; r + 2^q \;:\; r \in L \,].$$

Define $\mathcal{R}_1 = [\,1\,]$ and $\mathcal{R}_{q+1} = \mathrm{children}_q\big(\mathrm{filter}\,\mathrm{survives}(q, \cdot)\,\mathcal{R}_q\big)$.

$\mathcal{R}_q$ is exactly the program's `relevant_residues` at level q, and $\mathcal{S}_q = \mathrm{filter}\,\mathrm{survives}(q, \cdot)\,\mathcal{R}_q$ is `surviving_residues`.

Proposition 6.2 (Structural invariants)**.** *For every $q \geq 1$ and every $r \in \mathcal{R}_q$:*

(i) r is odd;

(ii) $1 \leq r < 2^q$, so r is the least positive member of its class modulo 2^q;

(iii) $\mathcal{R}_q$ has no repeated entries.

Proof. Induction on q. For $q = 1$, $\mathcal{R}_1 = [1]$ and $1 < 2$. (i) 2^q is even for $q \geq 1$, so r and $r + 2^q$ have the same parity. (ii) If $r < 2^q$ then both r and $r + 2^q$ lie below 2^{q+1}. (iii) Suppose $\mathcal{R}_q$ is repetition-free with all entries below 2^q. Within one parent r we have $r \neq r + 2^q$; across distinct parents $r \neq r'$, we have $r \neq r'$, $r + 2^q \neq r' + 2^q$, and $r \neq r' + 2^q$ because $r < 2^q \leq r' + 2^q$. □

Part (i) justifies (P1) at the level of the outer loop: every value fed to Eval is odd, so Proposition 3.1 applies. Part (ii) justifies (P3): the tested value really is the least member of its class, and by Corollary 4.4 that is the hardest case. Part (iii) is what makes the program's `len(set(surviving_residues))` equal to `len(surviving_residues)` — the `set()` is redundant but harmless — and it is what makes the reported density well defined.

Corollary 6.3 (The reported density)**.** *The quantity $100 - 100\,|\mathcal{S}_q|/2^q$ printed at level q is exactly the percentage of residue classes modulo 2^q that the sieve has proven to descend. All even classes are eliminated at $q = 1$, since $\mathrm{T}(2m) = m < 2m$, which is why the denominator is 2^q rather than the count 2^{q-1} of odd classes.*

Remark 6.4. Corollary 6.3 settles which of two natural quantities the program reports. Relative to the odd residues alone — the only ones the sieve ever refines — the surviving density is twice $|\mathcal{S}_q|/2^q$. Both are meaningful; the printed figure is the density among *all* integers.

Theorem 6.5 (Coverage)**.** *Let $n \geq 1$ be odd and let $q \geq 1$. Then at least one of the following holds:*

(a) there are $r \in \mathcal{R}_q$ and $j \geq 0$ with $n = r + j2^q$; or

(b) there are $q' \leq q$, $r \in \mathcal{R}_{q'}$ and $j \geq 0$ with $n = r + j2^{q'}$ and $\mathrm{T}^{q'}(r) \leq r$.

Proof. Induction on q. For $q = 1$: n is odd, so $n = 1 + j \cdot 2$ for some j, and $1 \in \mathcal{R}_1$, giving (a).

Assume the claim at q. If (b) holds, it holds verbatim at $q + 1$. Otherwise take $r \in \mathcal{R}_q$ and j with $n = r + j2^q$. If $\mathrm{survives}(q, r)$ fails, then $\mathrm{T}^q(r) \leq r$ and we are in case (b) with $q' = q$. If it holds, then $r \in \mathcal{S}_q$, so both r and $r + 2^q$ belong to $\mathcal{R}_{q+1}$. Split on the parity of j:

$$j = 2j' \;\Rightarrow\; n = r + j'2^{q+1}, \qquad j = 2j' + 1 \;\Rightarrow\; n = (r + 2^q) + j'2^{q+1},$$

which is case (a) at level $q + 1$. □

Theorem 6.6 (Main theorem: soundness of the sieve)**.** *Let $n \geq 1$ be odd, let $q \geq 1$, and suppose every m with $1 \leq m < n$ satisfies* reaches1(m)*. Then at least one of the following holds:*

(a) n's residue class is still present in $\mathcal{R}_q$ — the sieve makes no claim about n; or

(b) reaches1(n) *— so n is not a minimal counterexample; or*

(c) $\mathrm{T}^{q'}(n) = n$ for some $q' \leq q$, i.e. n lies on a cycle of the accelerated map.

Moreover, in case (c) the value n appears in the program's `cycles` *list.*

Proof. Apply Theorem 6.5. Case (a) is immediate. In case (b) of that theorem, we have $n = r + j2^{q'}$ with $r \geq 1$ and $\mathrm{T}^{q'}(r) \leq r$.

If $j \geq 1$, Theorem 5.4 yields reaches1(n). If $j = 0$ then $n = r$. Either $\mathrm{T}^{q'}(n) < n$, in which case $\mathrm{T}^{q'}(n)$ is a positive integer below n, so reaches1$(\mathrm{T}^{q'}(n))$ by hypothesis and hence reaches1(n); or $\mathrm{T}^{q'}(n) = n$, which is case (c).

For the final claim, observe that the coverage proof produces case (b) only with $r \in \mathcal{R}_{q'}$, and the program tests every element of $\mathcal{R}_{q'}$ at level q' and appends precisely those with $\mathrm{T}^{q'}(r) = r$ to `cycles`. Since $q' \leq q$, the value has already been recorded by the time level q is reported. □

Corollary 6.7. *If the sieve at level q discards a residue class, that class contains no minimal counterexample to the Collatz conjecture other than, possibly, a cycle element that the program reports.*

7 Rocq formalization

The development `proofs/CollatzSieve.v` formalizes everything above. Numbers are binary naturals (`N`); step counts are `nat`, so that iteration is structurally recursive.

```
Definition T (n : N) : N := if N.even n then n / 2 else (3 * n + 1) / 2.

Fixpoint Tp (k : nat) (n : N) : N :=
  match k with O => n | S k' => T (Tp k' n) end.

Definition reaches1 (n : N) : Prop := exists k, Tp k n = 1.
```

The inner loop is modelled with an explicit structural fuel argument, since $rem - d$ is not syntactically decreasing:

```
Fixpoint sim_aux (fuel rem : nat) (n : N) : N :=
  match fuel with
  | O   => n
  | S f => match rem with
           | O   => n
           | S _ => let y := 3 * n + 1 in
                    let d := Nat.min (v2 y) rem in
                    sim_aux f (rem - d) (y / pow2 d)
           end
  end.

Definition sim (q : nat) (n : N) : N := sim_aux q q n.
```

Since $d \geq 1$ at every iteration, fuel $= q$ always suffices; this is the content of the hypothesis $rem \leq fuel$ in `sim_aux_correct`.

The 2-adic valuation is defined by structural recursion on the binary representation, which makes it total and evaluation-friendly:

```
Fixpoint v2p (p : positive) : nat :=
  match p with xO q => S (v2p q) | _ => O end.

Definition v2 (y : N) : nat := match y with N0 => O | Npos p => v2p p end.
```

Result	Statement	Rocq name
Lemma 2.4	chained halving	`step_chain`
Prop. 3.1/3.2	Eval computes T^q	`sim_aux_correct`, `sim_correct`
–	`collatz_sim` spec	`collatz_sim_correct`
Lemma 4.2	$2^q T^q(n) = An + C$	`affine`, `A_odd`
Thm. 4.3	shift lemma (Terras)	`shift`, `shift_val`
Lemma 5.1	$A_q(r) < 2^q$	`A_lt_pow2`
Thm. 5.2	class-wide descent	`descent`
Thm. 5.4	no minimal counterexample	`no_minimal_counterexample`
Prop. 6.2	odd / bounded / distinct	`Rl_odd`, `Rl_range`, `Rl_nodup`
Thm. 6.5	coverage	`coverage`
Thm. 6.6	sieve soundness	`sieve_sound`
Thm. 6.6(c)	cycles are reported	`cycles_complete`

Table 1: Correspondence between the paper and `proofs/CollatzSieve.v`.

Table 1 maps the results of this paper to their Rocq names.

Two conveniences make the development robust. First, `AC` is unfolded exactly once, into recursion equations `Acoef_S` and `Ccoef_S`; every later proof rewrites with those rather than reducing `AC`, so no proof depends on how far `simpl` chooses to go. Second, `sim_aux` is given the equations `sim_aux_rem0` and `sim_aux_S` (both closed by `reflexivity`), because `simpl` would otherwise inline the `let`-bindings and expand $3n + 1$ into its binary representation.

Verification status. The file ends with `Print Assumptions` on every headline result. All eight report *Closed under the global context*: the development uses no axioms, no classical logic, no functional extensionality, and contains no `Admitted` proofs or `admit` tactics. It also contains executable sanity checks closed by `reflexivity`, for instance `Rl 1 = [1; 3]`, `survives 2 1 = false`, `Tp 2 1 = 1` and `sim 4 3 = 2`. Checked with the Rocq Prover 9.1.1 (OCaml 5.5.0); a clean build of all 742 lines takes under three seconds.

```
$ rocq compile proofs/CollatzSieve.v
Closed under the global context     (x8)
```

8 Computed data

Table 2 lists the program's output for $q \le 24$. The survivor counts reproduce the classical Terras sieve counts, and the residue sets agree elementwise with an independent step-by-step implementation of T that shares no code with the bit-twiddling version.

The only entry ever added to `cycles` is 1, contributed at $q = 2$ because $T(1) = 2$ and $T(2) = 1$. Class 1 mod 4 is therefore retired at level 2; by the remark following Theorem 5.2 every $n \equiv 1 \pmod 4$ with $n > 1$ satisfies $T^2(n) = (3n + 1)/4 < n$.

9 What is *not* proved

Theorem 6.6 is a soundness result about an elimination procedure. It is worth being explicit about its reach.

q	$\lvert\mathcal{S}_q\rvert$	eliminated (%)	q	$\lvert\mathcal{S}_q\rvert$	eliminated (%)
1	1	50.0000	13	367	95.5200
2	1	75.0000	14	734	95.5200
3	2	75.0000	15	1295	96.0480
4	3	81.2500	16	2114	96.7743
5	4	87.5000	17	4228	96.7743
6	8	87.5000	18	7495	97.1409
7	13	89.8438	19	14990	97.1409
8	19	92.5781	20	27328	97.3938
9	38	92.5781	21	46611	97.7774
10	64	93.7500	22	93222	97.7774
11	128	93.7500	23	168807	97.9877
12	226	94.4824	24	286581	98.2918

Table 2: Surviving residue classes and eliminated density by level.

1. **The sieve cannot prove the conjecture.** Terras [1] showed the surviving density tends to 0; Table 2 exhibits that decay. But a set of density zero can be infinite and nonempty, and the sieve provides no bound on the surviving classes' members. No finite run — and no limit of runs — settles Collatz.

2. **Cycle detection is incidental, not a search.** The equality branch fires only when a cycle's minimal element m satisfies $m < 2^L$ for its T-period L *and* m is still unsieved at level L. The program is not a cycle-finding algorithm, and Theorem 6.6(c) should not be read as one. What the theorem does guarantee is the converse direction that matters for soundness: no class is discarded on the strength of a fixed point without that fixed point being reported.

3. **Resource growth.** $|\mathcal{R}_q| = 2|\mathcal{S}_{q-1}|$ grows roughly geometrically (empirically by a factor near 1.7 per level), so memory is exponential in q. Since the program appends the full survivor list to its log at every level, log size is quadratic in the number of levels unless suppressed — which is what the $q \leq 15$ cutoff on line 8 of Algorithm 2 is for. No information is lost by that cutoff: $|\mathcal{S}_q|$ is recoverable from the reported percentage p as $|\mathcal{S}_q| = (100 - p)\, 2^q/100$.

4. **Machine integers.** The Rocq development reasons about mathematical integers. Python's `int` is arbitrary-precision, so this is faithful; a port to fixed-width arithmetic would need an overflow argument that is absent here.

10 Conclusion

The three suspicious-looking features of `Residual_Analysis.py` are all correct, and each for a specific reason. The unconditional $3m + 1$ is safe because oddness is a loop invariant of both loops (Propositions 3.1 and 6.2(i)). The truncated shift computes $\mathrm{T}^q(n)$ on the nose, because a truncated division is the tail of the last T-step rather than a corruption of it (Proposition 3.2). And testing the least representative decides the entire class, because T^q is affine on each class with an odd multiplier (Theorem 4.3, Lemma 5.1).

The one defect the analysis surfaced was not in the arithmetic but in the reporting: the `cycles` list was populated and never printed. Since Theorem 6.6(c) identifies that list as precisely the set of exceptions to the soundness statement, the program's most consequential possible output was the one it could not emit. That has been corrected.

References

[1] R. Terras. A stopping time problem on the positive integers. *Acta Arithmetica*, 30(3):241–252, 1976.

[2] C. J. Everett. Iteration of the number-theoretic function $f(2n) = n$, $f(2n+1) = 3n+2$. *Advances in Mathematics*, 25(1):42–45, 1977.

[3] J. C. Lagarias. The $3x+1$ problem and its generalizations. *The American Mathematical Monthly*, 92(1):3–23, 1985.

[4] J. C. Lagarias, editor. *The Ultimate Challenge: The* $3x+1$ *Problem*. American Mathematical Society, 2010.

[5] T. Tao. Almost all orbits of the Collatz map attain almost bounded values. *Forum of Mathematics, Pi*, 10:e12, 2022.

[6] The Rocq Development Team. *The Rocq Prover*. https://rocq-prover.org/.

[7] RobinCodes. Collatz — residual analysis and orbital graphs. https://github.com/RobinCodes/Collatz.